ENTERPRISE DIGITAL BANKING SYSTEM

01

Architecture Documentation

System architecture, component design, deployment architecture, technology choices and Architecture Decision Records (ADRs).

Document	01 — Architecture Documentation
System	Enterprise Digital Banking System (banking-app) + notification-service
Stack	Java 17 · Spring Boot 4.1 · Spring Security + JWT · Spring Data JPA · MySQL 8 · Apache Kafka 3.8 · Redis 7 · Docker
Repository	Enterprise-Digital-Banking-System (branch `master`)
Version / date	1.0 — 13 October 2026
Author	hemantwint-hey

Status tags used throughout: **BUILT** exists in the code today **PLANNED** part of the target design (roadmap docs 01–09), not yet implemented **GAP** known limitation

Contents

1. 1. System Overview
2. 2. System Architecture Diagram
3. 3. Layered Architecture
4. 4. Component Diagram
5. 5. Event-Driven Integration
6. 6. Deployment Architecture
7. 7. Technology Choices
8. 8. Architecture Decision Records
9. 9. Quality Attributes

1. System Overview

The **Enterprise Digital Banking System** is a backend platform that simulates the core operations of a retail bank: user registration and login, customer onboarding with KYC, savings/current accounts, deposits, withdrawals and transfers backed by a **double-entry ledger**, saved beneficiaries, loans with an EMI repayment schedule, and a full **audit trail**. Customer-facing notifications are handled by a separate **notification-service** that consumes domain events from Apache Kafka.

1.1 Goals

- **Correctness of money movement** — every balance change is recorded atomically as a transaction plus ledger entries; debits always equal credits.
- **Security** — stateless JWT authentication, BCrypt password hashing, no secrets in source control.
- **Traceability** — every important state change is written to `audit_logs` with who, what, old/new value and client IP.
- **Loose coupling** — notifications are asynchronous; a notification outage can never fail or slow a transfer.
- **Operability** — containerised, observable (Actuator, Prometheus, Grafana) and deployable to AWS.

1.2 Actors

Actor	Description	Typical actions
Customer	Retail bank user (role `CUSTOMER`)	Register, log in, view accounts, transfer money, manage beneficiaries, apply for loans, pay EMIs
Employee / Manager	Bank staff (roles `EMPLOYEE`, `MANAGER`)	Onboard customers, complete KYC, open/freeze/close accounts, approve or reject loans
Admin	Platform administrator (role `ADMIN`)	User management, configuration, everything staff can do
Auditor	Compliance user (role `AUDITOR`)	Read-only access to audit logs and ledgers
notification-service	Internal system actor	Consumes Kafka events and delivers e-mail / SMS / push / in-app notifications

> **Role enforcement:** the five roles exist in the `Role` enum and are placed in the JWT principal's authorities, but endpoint-level authorization (`@PreAuthorize` / `requestMatchers(...).hasRole(...)`) is **PLANNED**. Today every authenticated user can call every `/api/v1/**` endpoint. See document 05 — Security.

1.3 Scope

Capability	Status
Authentication (register, login, JWT)	**BUILT**
Customer profile + KYC (PAN / Aadhaar)	**BUILT**
Accounts (open, update, freeze, close)	**BUILT**
Transactions (deposit, withdraw, transfer) + double-entry ledger	**BUILT**

Beneficiaries (add, activate, transfer-by-beneficiary)	**BUILT**
Loans + EMI schedule	**BUILT**
Audit logging	**BUILT**
Kafka events → notification-service (e-mail, SMS, push, in-app)	**BUILT** (SMTP credentials deferred)
Cards	**PLANNED** (package skeleton only)
Redis cache, pagination, search, Swagger	**PLANNED** (roadmap 01, 06)
Docker Compose, profiles, Flyway, Actuator/Prometheus/Grafana, Nginx + rate limiting, AWS	**PLANNED** (roadmap 03, 04, 08, 09)
Refresh tokens, forgot/reset/change password	**PLANNED** (DTOs exist, no endpoints)

2. System Architecture Diagram

The target architecture (after roadmap items 01–09). Components marked *planned* are not yet in the running system; the core path Client → banking-app → MySQL / Kafka → notification-service exists today.

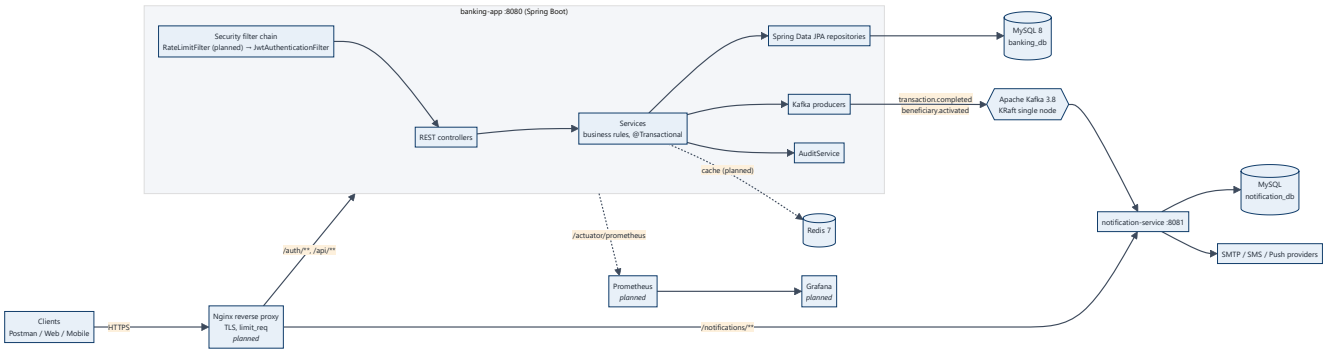


Figure 1 — System architecture (target state)

2.1 Runtime components

Component	Responsibility	Port	Status
banking-app	All banking REST APIs, business rules, persistence, event publishing, audit	8080	BUILT
notification-service	Kafka consumers; notification engine (strategy, decorator, observer, retry with exponential back-off, idempotency key)	8081	BUILT
MySQL 8	`banking_db` and `notification_db` (separate schemas, one per service)	3306	BUILT
Apache Kafka 3.8 (KRaft)	Event bus between the two services	9092	BUILT (compose file in notification-service)
Redis 7	Account read cache; shared rate-limit counters	6379	PLANNED
Nginx	Single public entry point, TLS termination, per-IP rate limiting, body-size limits	80/443	PLANNED
Prometheus + Grafana	Metrics scraping and dashboards	9090 / 3000	PLANNED

3. Layered Architecture

banking-app is a **modular monolith**: one deployable, organised **package-by-feature** (`account` , `customer` , `transaction` , …). Inside every feature the same four layers repeat.

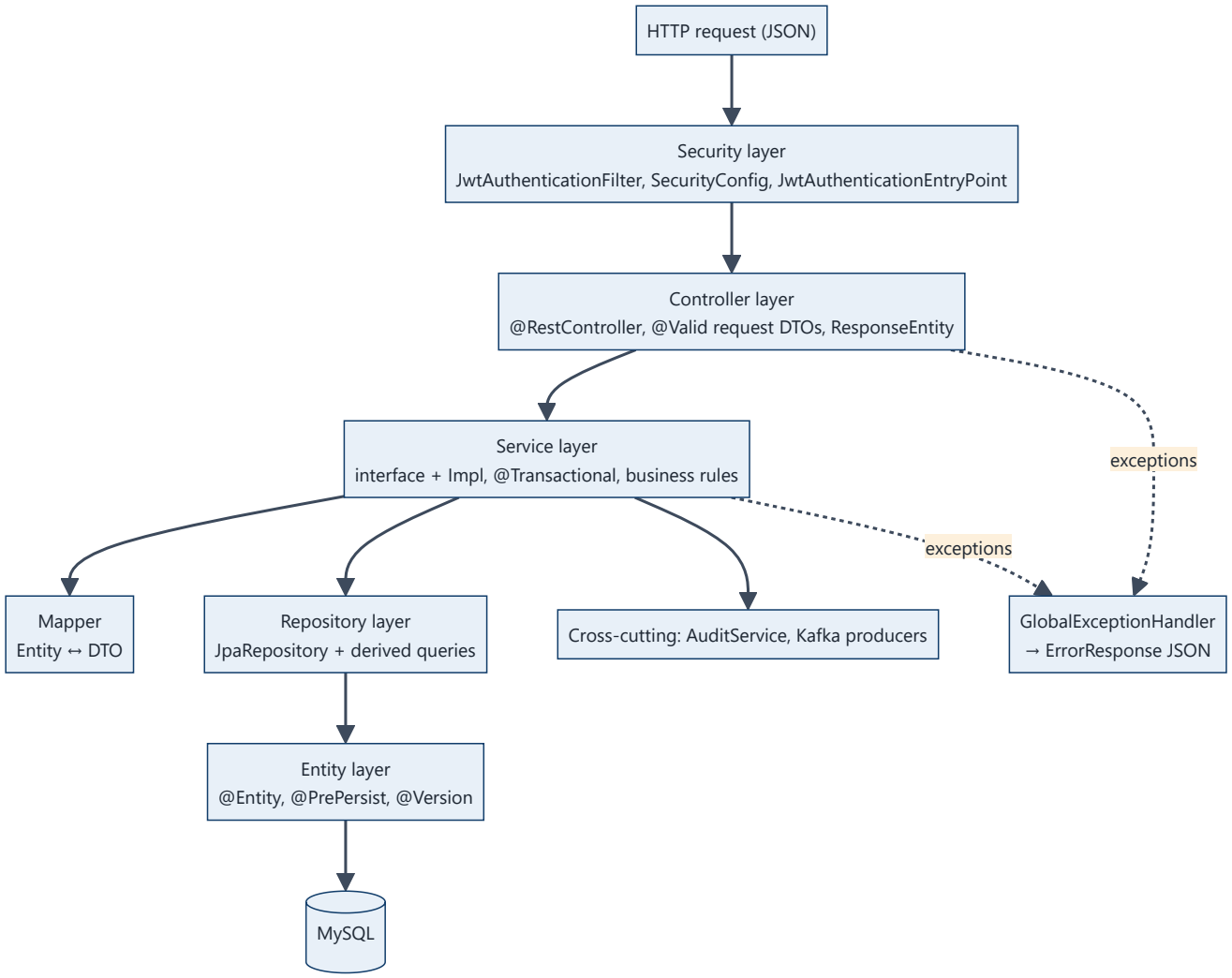


Figure 2 — Request path through the layers

Layer	Rules
Controller	Thin. Validates input with Bean Validation (`@Valid`), delegates to exactly one service call, never touches repositories, returns DTOs only (never entities).
Service	Owns business rules and the transaction boundary (`@Transactional` at class level, `readOnly = true` on queries). Throws domain exceptions (`BadRequestException` , `ResourceNotFoundException` , …).
Mapper	Hand-written `@Component` classes converting entities to response DTOs (and request DTOs to entities). No business logic.
Repository	Spring Data JPA interfaces; derived query methods only (parameterised SQL, no string concatenation).
Entity	JPA mapping, defaults via `@PrePersist` , enums stored as `STRING` , money as `DECIMAL(18,2)` , optimistic locking on `Account` via `@Version` .

3.1 Package structure

```
org.example.enterprisedigitalbankingsystem
├── EnterpriseDigitalBankingSystemApplication.java
├── auth/          controller · dto · entity(User, Role, UserStatus) · repository · service
├── customer/      controller · dto · entity(Customer, KYCStatus) · mapper · repository · service
├── account/       controller · dto · entity(Account, AccountType, AccountStatus) · mapper · repository ·
service
├── transaction/   controller · dto · entity(Transaction, TransactionType, TransactionStatus) · mapper ·
repository · service
├── ledger/        controller · dto · entity(LedgerEntry, EntryType) · mapper · repository · service
├── beneficiary/   controller · dto · entity(Beneficiary, BeneficiaryStatus) · mapper · repository · service
├── loan/          controller · dto · entity(Loan, LoanType, LoanStatus) · mapper · repository · service
├── emi/           controller · dto · entity(Emi, EmiStatus) · mapper · repository · service
├── audit/         controller · dto · entity(AuditLog, AuditAction) · mapper · repository · service
├── card/          skeleton only (PLANNED)
├── kafka/         config(KafkaProducerConfig) · event(*Event) · producer(*EventProducer)
├── security/      JwtUtil · JwtAuthenticationFilter · JwtAuthenticationEntryPoint · SecurityConfig ·
CustomUserDetails(Service)
├── config/        SecurityBeansConfig (BCryptPasswordEncoder)
└── exception/     GlobalExceptionHandler · ErrorResponse · custom exceptions
```

4. Component Diagram

Service-level dependencies inside banking-app. Arrows mean "calls / depends on". Note that services may depend on *another module's repository*, but service-to-service dependencies only ever point one way (no cycles).

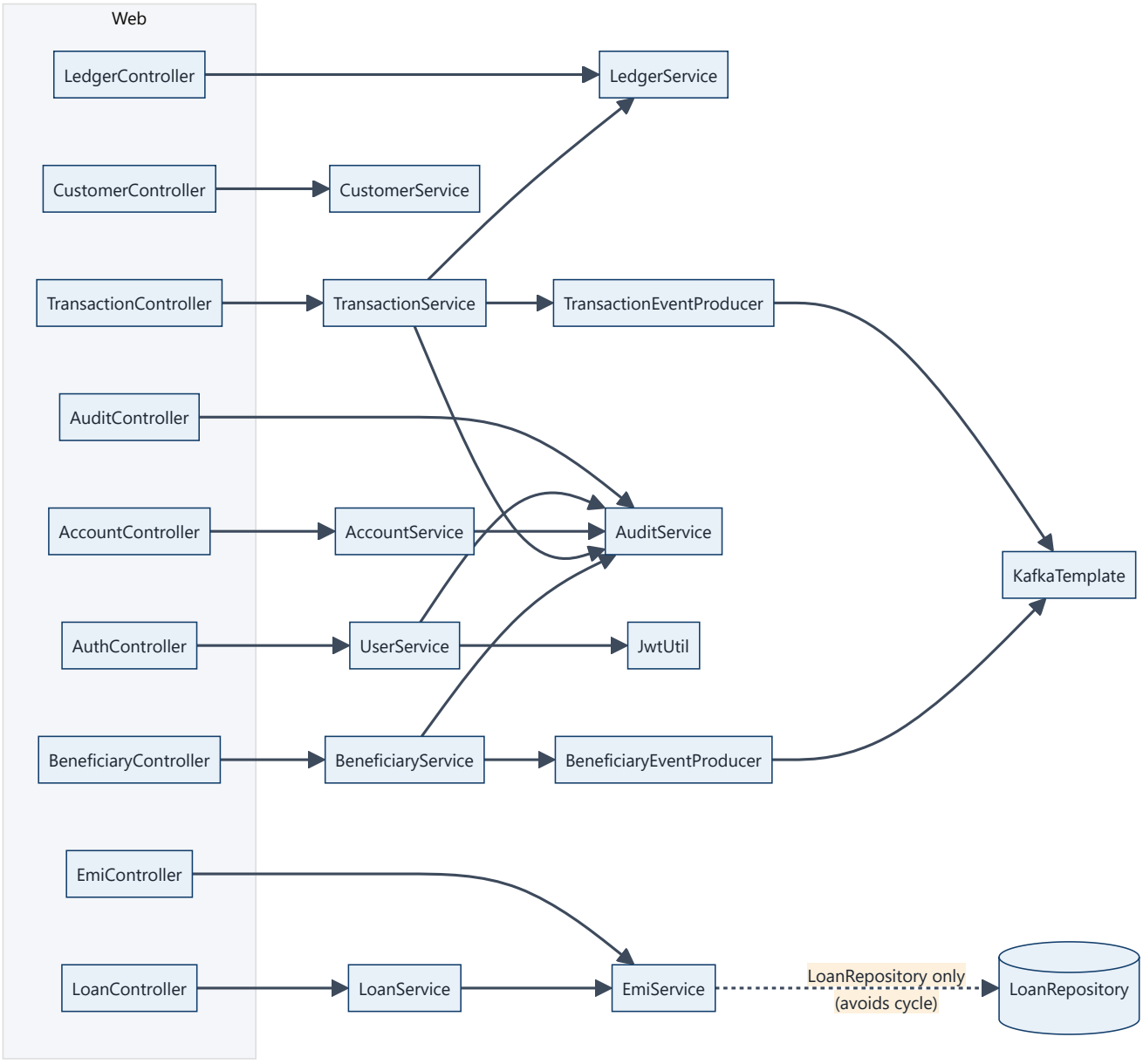


Figure 3 — Component / service dependency diagram

4.1 Module catalogue

Module	Owns table(s)	Depends on	Publishes / side effects
auth	`users`	UserRepository, PasswordEncoder, JwtUtil, AuditService	Audit: CREATE, LOGIN, LOGIN_FAILED
customer	`customers`	CustomerRepository, UserRepository	Sets `users.kyc_verified` on KYC
account	`accounts`	AccountRepository, CustomerRepository, AuditService	Audit: CREATE/UPDATE

transaction	`transactions`	AccountRepository, BeneficiaryRepository, LedgerService, AuditService, TransactionEventProducer	Ledger entries; Kafka `transaction.completed` (transfers); audit
ledger	`ledger_entries`	LedgerRepository	—
beneficiary	`beneficiaries`	BeneficiaryRepository, CustomerRepository, AccountRepository, AuditService, BeneficiaryEventProducer	Kafka `beneficiary.activated`; audit
loan	`loans`	LoanRepository, CustomerRepository, AccountRepository, EmiService	Credits disbursement account on approval; generates EMI schedule
emi	`emis`	EmiRepository, LoanRepository	Closes loan when last EMI is paid
audit	`audit_logs`	AuditRepository, SecurityContext, current HTTP request	—
card	`cards` (planned)	—	PLANNED

4.2 notification-service internals

notification-service is a separate Spring Boot application (port 8081) with its own database. Its design uses classic patterns:

Pattern	Classes	Purpose
Strategy	`NotificationChannel` → Email / Sms / Push / InApp; `NotificationChannelResolver`	Pick the delivery channel by `NotificationType`
Decorator	`SimpleNotification`, `TimestampNotificationDecorator`, `SignatureNotificationDecorator`	Compose message content
Observer	`NotificationEventPublisher` + Audit / Logging / Metrics listeners	React to lifecycle events (sent, failed, retrying)
Retry policy	`ExponentialBackoffRetryPolicy`	Retry failed deliveries with exponential back-off
Idempotency	Unique `idempotency_key` = event's `eventId`	A redelivered Kafka message never sends a duplicate notification

5. Event-Driven Integration

Topic	Producer	Trigger	Message key	Consumer (group `notification-service`)
`transaction.completed`	TransactionEventProducer	Successful **transfer**	source `accountId`	TransactionEventConsumer → HIGH-priority e-mail
`beneficiary.activated`	BeneficiaryEventProducer	Beneficiary moved PENDING → ACTIVE	`customerId`	BeneficiaryEventConsumer → MEDIUM-priority e-mail
`<topic>.DLT`	DeadLetterPublishingRecoverer	Consumer failed 3 times (2 s apart)	same	Manual inspection **PLANNED**

- Payloads are JSON strings (`StringSerializer`) built with Jackson. Keying by account / customer keeps all events for one account in order on one partition.
- Publishing is **fire-and-forget**: producer exceptions are logged and swallowed so a Kafka outage never rolls back a committed money movement.
- The event is published inside the service's `@Transactional` method, before commit. **GAP** if the DB commit later fails, a notification can be sent for a transfer that did not happen. A transactional outbox (or `@TransactionalEventListener(AFTER_COMMIT)`) is the documented fix.

6. Deployment Architecture

6.1 Local / Docker Compose PLANNED

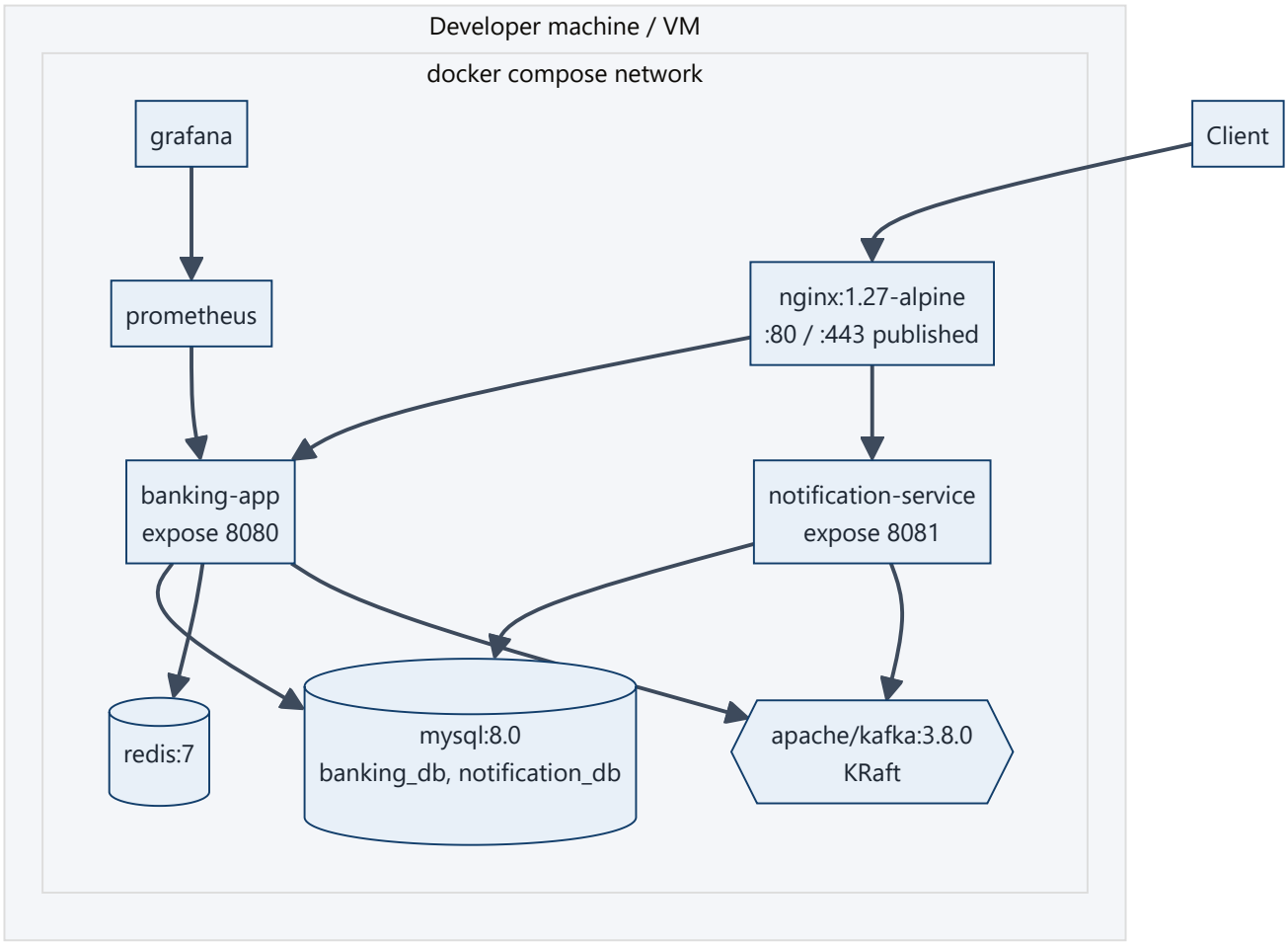


Figure 4 — Container topology (target). Only Nginx is published to the host.

6.2 Cloud (AWS) PLANNED

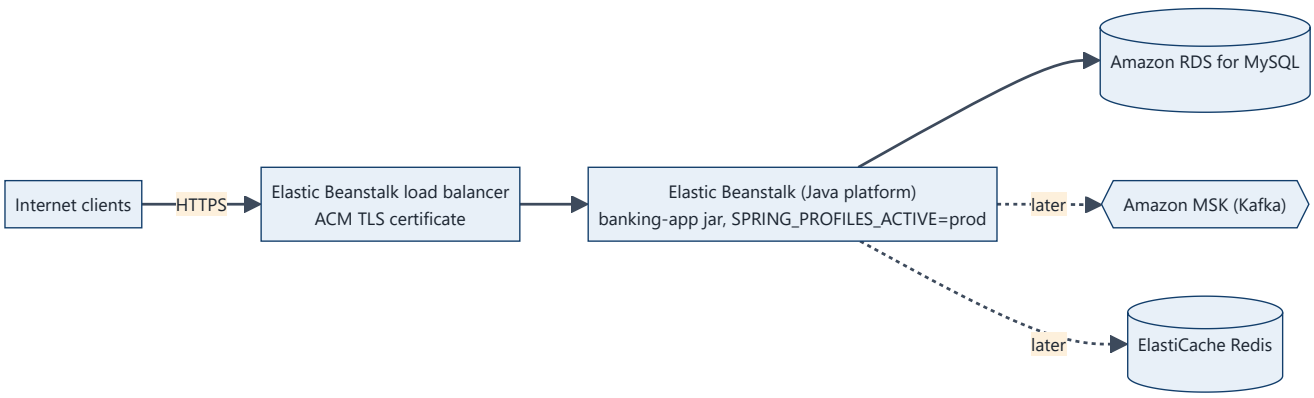


Figure 5 — Minimal AWS deployment: app + RDS first; Kafka/Redis remain containerised and documented.

7. Technology Choices

Concern	Choice	Why
Language / runtime	Java 17 (LTS)	Records, pattern matching for `instanceof`, long-term support
Framework	Spring Boot 4.1 (webmvc, data-jpa, security, validation)	Industry standard for enterprise Java; auto-configuration
Persistence	Spring Data JPA / Hibernate + MySQL 8	Relational integrity for money; ACID transactions; `DECIMAL` precision
Authentication	JWT (jjwt 0.12.6, HS256) + BCrypt	Stateless, horizontally scalable; no server sessions
Messaging	Apache Kafka 3.8 (KRaft, no ZooKeeper)	Durable, ordered, replayable events; decouples notifications
Boilerplate	Lombok	Builders, getters/setters, constructor injection
Caching	Redis 7 via Spring Cache **PLANNED**	Fast account reads with TTL safety net
Migrations	Flyway **PLANNED**	Versioned, repeatable schema changes
Observability	Logback, Actuator, Micrometer, Prometheus, Grafana **PLANNED**	Health, metrics, dashboards
API docs	springdoc-openapi 2.6 **PLANNED**	Live Swagger UI generated from code
Edge	Nginx + Bucket4j / Redis rate limiter **PLANNED**	TLS, single entry point, brute-force protection
Build	Maven wrapper (`mvnw`)	Reproducible builds without a global Maven install

8. Architecture Decision Records

Each ADR records one significant decision: context, decision, and consequences. Status values: *Accepted*, *Proposed*, *Superseded*.

ADR-001 — Modular monolith, package-by-feature Accepted

Context: one developer, many related banking domains that share transactions (a transfer touches accounts, ledger, audit).
Decision: a single Spring Boot app with one package per feature, each with controller/service/repository/entity sub-packages.
Consequences: simple local ACID transactions across modules; modules can later be extracted along the same seams. Discipline needed to avoid cross-module cycles.

ADR-002 — Notifications in a separate service via Kafka Accepted

Context: sending e-mail is slow and unreliable; a failed e-mail must never fail a transfer.
Decision: remove the embedded notification module; banking-app publishes events to Kafka, notification-service consumes them.
Consequences: transfers stay fast and independent of SMTP; notifications are eventually consistent; two services, two databases, one more piece of infrastructure (Kafka).

ADR-003 — Double-entry ledger alongside transactions Accepted

Context: a single `transactions` row cannot express per-account movements cleanly, and audits need a balance trail.
Decision: every transaction writes `LedgerEntry` rows (DEBIT/CREDIT) with `balance_after_entry`. A transfer writes exactly one DEBIT and one CREDIT of equal amount.
Consequences: statement generation and reconciliation become trivial; extra writes per transaction.

ADR-004 — Child references parent, no bidirectional collections Accepted

Context: `@OneToMany` collections invite N+1 queries and accidental cascade deletes.
Decision: children hold `@ManyToOne(fetch = LAZY)` to the parent (`Emi → Loan`, `LedgerEntry → Transaction`, `Account → Customer`); parents stay thin; children are fetched with repository queries.
Consequences: predictable SQL; slightly more repository methods.

ADR-005 — Break service cycles by depending on repositories Accepted

Context: `LoanService` needs `EmiService` (generate schedule) and EMI payment must close the loan.
Decision: `EmiServiceImpl` depends on `LoanRepository`, never on `LoanService`.
Consequences: no circular bean dependency; loan status rules for closing live in the EMI service.

ADR-006 — Stateless JWT authentication Accepted

Decision: HS256-signed JWT, subject = e-mail, 24 h expiry; `SessionCreationPolicy.STATELESS`; CSRF disabled (no cookies).
Consequences: scales horizontally; tokens cannot be revoked before expiry until refresh tokens / a deny-list are added.

ADR-007 — Money as BigDecimal / DECIMAL(18,2) Accepted

Decision: all amounts are `BigDecimal` in Java and `DECIMAL(18,2)` in MySQL. The EMI formula uses `double` only for `(1+r)^n` and rounds immediately with `HALF_UP`.
Consequences: no floating-point drift in stored balances.

ADR-008 — Optimistic locking on Account Accepted

Decision: `@Version` column on `accounts`. Concurrent writers to the same account cause an `ObjectOptimisticLockingFailureException` instead of a lost update.
Consequences: correct under concurrency; clients may need to retry (currently surfaces as HTTP 500 — mapping to 409 is PLANNED).

ADR-009 — Keep MySQL (not PostgreSQL) Accepted

Decision: the roadmap mentioned PostgreSQL for later days; the project has used MySQL from day 1, so it stays on MySQL 8.
Consequences: no migration risk; Flyway uses `flyway-mysql`.

ADR-010 — Schema owned by Flyway in production Proposed

Decision: capture the Hibernate-generated schema as `V1__baseline.sql`, add indexes in `V2__indexes.sql`, and run prod with `ddl-auto=validate`.
Consequences: reproducible schema; every change requires a migration file.

ADR-011 — Cache-aside with full eviction for accounts Proposed

Decision: `@Cacheable("accounts")` on reads by id and by number; `@CacheEvict(allEntries = true)` on every write (account updates *and* deposit / withdraw / transfer), 10-minute TTL.
Consequences: never serves a stale balance as long as every write path evicts; TTL bounds the damage of a missed eviction.

ADR-012 — Two-layer rate limiting (Nginx + app filter) Proposed

Decision: Nginx `limit_req` per IP at the edge; `RateLimitFilter` per user / endpoint in the app (Bucket4j in-memory first, Redis Lua fixed-window for multi-instance).
Consequences: brute-force protection on `/auth/login`; 429 responses with `Retry-After`.

9. Quality Attributes

Attribute	How the architecture addresses it
Consistency	Service-level `@Transactional`: balance update, transaction row, ledger entries and audit log commit or roll back together.
Concurrency safety	`@Version` optimistic locking on accounts.
Availability	Notifications decoupled through Kafka; producer failures are swallowed and logged.
Security	JWT, BCrypt, env-var secrets, generic 500 messages, parameterised queries. See doc 05.
Performance	Lazy associations, read-only transactions, Redis cache and pagination (planned), explicit indexes (planned).
Maintainability	Consistent package-by-feature layout, DTO/mapper separation, centralised exception handling.
Observability	Audit log today; Actuator + Prometheus + Grafana planned.